

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

отчет по блоку задач №1 (Яндекс.Контест)

Выполнил:

Студент группы Р3210

Бармичев Григорий Андреевич

Преподаватель:

Косяков Михаил Сергеевич

Санкт-Петербург, 2026г.

Задача №1 «А. Агроном-любитель»

Описание:

Дана последовательность из n цветов. Нужно найти самый длинный участок, где нет трех одинаковых цветов подряд.

Доказательство:

Используем скользящее окно: двигаем правую границу r слева направо и поддерживаем левую границу $current_l$, чтобы отрезок $[current_l, r]$ оставался допустимым. Единственное нарушение возможно, когда на конце появляется тройка $a_r = a_{r-1} = a_{r-2}$. Тогда достаточно сдвинуть левую границу на $r - 1$, убрав элемент $r - 2$. На каждом r считаем длину текущего окна и запоминаем лучший ответ.

Алгоритмическая сложность:

По времени: проходит массив один раз, а все проверки и возможный сдвиг выполняются за $O(1)$ на шаг. Поэтому временная сложность равна $O(n)$.

По памяти: хранится массив a длины n и несколько переменных. Пространственная сложность равна $O(n)$.

Код:

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int n;
    if (!(cin >> n)) {
        return 0;
    }

    vector<int> a(n);
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }

    if (n <= 2) {
        cout << 1 << " " << n << endl;
        return 0;
    }

    int best_l = 0;
    int best_r = 0;
    int max_len = 0;
    int current_l = 0;

    for (int r = 0; r < n; ++r) {
        if (r >= 2 && a[r] == a[r - 1] && a[r] == a[r - 2]) {
            current_l = r - 1;
        }

        int current_len = r - current_l + 1;
        if (current_len > max_len) {
            max_len = current_len;
        }
    }

    cout << max_len << endl;
    return 0;
}
```

```

    best_l = current_l;
    best_r = r;
}
}

cout << best_l + 1 << " " << best_r + 1 << endl;

return 0;
}

```

Задача №2 «В. Зоопарк Глеба»

Описание:

По окружности в заданном порядке стоят $2n$ объектов: маленькая буква — животное, соответствующая большая — его ловушка. Нужно понять, можно ли соединить каждого зверя с его ловушкой хордами так, чтобы хорды не пересекались, если можно — вывести для каждой ловушки номер животного, которое в неё попадёт.

Доказательство:

Задача решается через стек. Мы перебираем символы строки по порядку. Если стек пуст, мы просто кладем в него текущий элемент (тип и ID). Если стек не пуст, мы сравниваем текущий символ с тем, что лежит на вершине стека.

Если это один и тот же символ, но в разном регистре, значит, мы нашли пару (животное и его ловушку). Записываем связь в массив result по индексу ловушки записываем ID соответствующего животного. Затем удаляем верхний элемент из стека, так как эта пара больше не мешает другим. В случае, когда не пара: Мы кладем текущий символ в стек. Он становится новым верхним элементом и будет ждать свою пару.

Если мы встречаем пару для не верхнего элемента, в то время как сверху всё еще лежит другой символ, это свидетельствует о чередовании их позиций на окружности, что неизбежно приведет к пересечению путей.

Таким образом, если после прохода по всей строке стек оказался пуст — значит, все нашли свои пары и пересечений нет.

Алгоритмическая сложность:

По времени: Программа совершает один цикл по строке длиной $2n$, а все операции со стеком занимают константное время. Временная сложность равна $O(n)$.

По памяти: В худшем случае, когда все пары являются вложенными, стек может вырасти до размера всей входной последовательности $2n$. Пространственная сложность равна $O(n)$.

Код:

```

#include <cctype>
#include <iostream>
#include <stack>
#include <string>
#include <vector>

using namespace std;

struct Element {
    char type;

```

```

int id;
};

bool IsPair(char a, char b) {
    if (tolower(a) != tolower(b)) {
        return false;
    }
    return a != b;
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    string s;
    if (!(cin >> s)) {
        return 0;
    }

    int n = s.length() / 2;
    vector<int> result(n + 1);
    stack<Element> st;

    int animal_count = 0;
    int trap_count = 0;

    for (char c : s) {
        int current_id;
        bool is_animal = (c >= 'a' && c <= 'z');

        if (is_animal) {
            current_id = ++animal_count;
        } else {
            current_id = ++trap_count;
        }

        if (!st.empty() && IsPair(st.top().type, c)) {
            if (is_animal) {
                result[st.top().id] = current_id;
            } else {
                result[current_id] = st.top().id;
            }
            st.pop();
        } else {
            st.push({c, current_id});
        }
    }

    if (st.empty()) {
        cout << "Possible" << endl;
        for (int i = 1; i <= n; ++i) {
            cout << result[i] << (i == n ? "" : " ");
        }
        cout << endl;
    }
}

```

```
} else {  
    cout << "Impossible" << endl;  
}  
  
return 0;  
}
```

Задача №3 «С. Конфигурационный файл»

Описание:

Нужно обработать конфигурационный файл, в котором задаются переменные и их значения, а также имеются блоки (уровни вложенности).

Доказательство:

Для каждой переменной в хеш-таблице хранится только одно, самое актуальное на данный момент значение. Когда программа встречает операцию присваивания, она сначала сохраняет в стек текущее имя переменной и её старое значение, а уже затем записывает в таблицу новое. Если переменной присваивается значение другой переменной, программа берет его из таблицы.

При входе в новый блок программа добавляет старые значения в стек истории, а когда блок заканчивается, начинает разматывать его в обратном порядке.

Алгоритмическая сложность:

N - количество строк во входных данных, L - максимальная длина имени переменной

По времени: Каждая строка обрабатывается один раз. Операции поиск и вставка в среднем занимают $O(L)$, так как нужно вычислить хеш от строки. Временная сложность равна $O(NL)$.

По памяти: Мы храним все текущие переменные в хеш-таблице и историю всех изменений в векторе. В худшем случае (если нет закрывающихся скобок) мы запишем в память каждое присваивание. Пространственная сложность равна $O(NL)$.

Код:

```
#include <iostream>  
#include <string>  
#include <unordered_map>  
#include <vector>  
  
using namespace std;  
  
int main() {  
    unordered_map<string, int> current_values;  
    vector<pair<string, int>> history;  
  
    string line;  
    while (cin >> line) {  
        if (line == "{") {  
            history.push_back({"", 0});  
        } else if (line == "}") {  
            while (!history.empty() && history.back().first != "{") {  
                current_values[history.back().first] = history.back().second;  
                history.pop_back();  
            }  
        }  
    }  
}
```

```

    if (!history.empty()) {
        history.pop_back();
    }
} else {
    size_t pos = line.find('=');
    string var_left = line.substr(0, pos);
    string right = line.substr(pos + 1);

    int new_val;
    if (isdigit(right[0]) || right[0] == '-') {
        new_val = stoi(right);
    } else {
        new_val = current_values[right];
        cout << new_val << "\n";
    }

    history.push_back({var_left, current_values[var_left]});
    current_values[var_left] = new_val;
}
}

return 0;
}

```

Задача №4 «D. Профессор Хаос»

Описание:

Каждый день количество бактерий умножается на b и уменьшается на c . Если после этого остаётся меньше c бактерий, эксперимент завершается. Если количество бактерий превышает d , оно ограничивается этим значением.

Доказательство:

Алгоритм пошагово имитирует каждый день эксперимента. Сначала количество бактерий увеличивается в b раз, затем из них вычитается число c . Если результат становится нулевым или отрицательным, эксперимент немедленно прекращается. Далее применяется ограничение контейнера: если бактерий больше d , их число приравнивается к d . Если после всех операций количество бактерий осталось таким же, как в начале дня, цикл прерывается, так как функция монотонна в последующие дни результат уже не изменится.

Алгоритмическая сложность:

По времени: хотя цикл идет до k , из-за ограниченного диапазона значений, цикл совершит не более $d + 1$ итерации. Временная сложность равна $O(\min(k, d))$.

По памяти: хранится несколько целых переменных, дополнительных структур нет.

Пространственная сложность равна $O(1)$.

Код:

```

#include <iostream>

using namespace std;

int main() {

```

```
long long a, b, c, d, k;
if (!(cin >> a >> b >> c >> d >> k)) {
    return 0;
}

long long current = a;

for (long long i = 1; i <= k; ++i) {
    long long next_val = current * b - c;

    if (next_val <= 0) {
        current = 0;
        break;
    }

    if (next_val > d) {
        next_val = d;
    }

    if (next_val == current) {
        break;
    }

    current = next_val;
}

cout << current << endl;

return 0;
}
```